

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Задача минимизации задержек

Студент гр. 4342	_____	Е.Н. Озернов
Студент гр. 4342	_____	М.Н. Митюков
Студент гр. 4342	_____	Г.А. Алюкин
Руководитель	_____	Т.Р. Жангиров

Санкт-Петербург
2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Е.Н. Озернов

Студент: М.Н. Митюков

Студент: Г.А. Алюкин

Группа: 4342

Тема практики: Задача минимизации задержек

Задание на практику:

Дано N задач, каждая из которых имеет свое время выполнения и дедлайн к которому она должна быть выполнена. Задача, это составить расписание с минимальными задержками. Задержка – количество времени на которое выполнение задач превысило дедлайны.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 08.07.2026

Дата защиты отчета: 00.00.0000

Студент гр. 4342

Студент гр. 4342

Студент гр. 4342

Руководитель

Е.Н. Озернов

М.Н. Митюков

Г.А. Алюкин

Т.Р. Жангиров

АННОТАЦИЯ

Отчёт посвящён разработке приложения с графическим интерфейсом для решения задачи минимизации суммарной задержки при планировании работ с помощью генетического алгоритма. Описана доменная модель приложения: представление входных задач (время выполнения и дедлайн), настраиваемые параметры алгоритма, а также структура состояний популяции, сохраняемых по каждому поколению, изложена реализация генетического алгоритма: кодирование решений в виде перестановок задач, операторы селекции (турнирный отбор), скрещивания (Order Crossover) и мутации (swap mutation), формирование функции приспособленности и механизм ранней остановки, архитектура GUI с пошаговой визуализацией поиска решения и графиком сходимости, а также результаты тестирования программы.

SUMMARY

The report is devoted to the development of an application with a graphical interface to solve the problem of minimizing the total delay in work planning using a genetic algorithm. The domain model of the application is described: the representation of input tasks (execution time and deadline), configurable algorithm parameters, as well as the structure of population states stored for each generation, the implementation of the genetic algorithm is described: encoding solutions in the form of permutations of tasks, selection operators (tournament selection), crosses (Order Crossover) and mutations (swap mutation), the formation of the fitness function and the mechanism of early stopping, GUI architecture with step-by-step visualization of the solution search and convergence graph, as well as the results of testing the program.

СОДЕРЖАНИЕ

Оглавление

ВВЕДЕНИЕ.....	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	7
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	8
2.1. Изучение постановки задачи минимизации суммарной задержки	8
2.2. Изучение принципов работы генетических алгоритмов.....	8
2.3. Спроектировать структуру данных для задач, параметров алгоритма и состояний популяции по поколениям	9
2.4. Реализовать генетический алгоритм самостоятельно, с настраиваемыми пользователем параметрами.....	11
2.5. Разработка графического интерфейса с вводом данных вручную, из файла и случайной генерацией	12
2.6. Реализация пошаговой визуализации процесса поиска решения	13
2.7. Реализация графика изменения качества решения по поколениям	14
2.8. Протестировать алгоритм и программу на разных наборах данных	14
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	17
4 ОТЗЫВ РУКОВОДИТЕЛЯ	19
ЗАКЛЮЧЕНИЕ	20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	21
ПРИЛОЖЕНИЕ А	22

ВВЕДЕНИЕ

Предметная область практики. Практика посвящена разработке программного обеспечения для решения задач теории расписаний с применением генетических алгоритмов. Рассматривается задача минимизации задержек при выполнении набора работ с заданными временами выполнения и дедлайнами — классическая NP-трудная задача комбинаторной оптимизации, для решения которой применяются приближённые методы, в том числе генетические алгоритмы.

Цель практики. Целью практики является разработка программного приложения с графическим пользовательским интерфейсом, реализующего самостоятельно написанный эволюционный алгоритм для решения задачи минимизации задержек, а также обеспечивающего наглядную пошаговую визуализацию процесса поиска оптимального решения на протяжении работы алгоритма.

Задачи практики. Изучить постановку задачи минимизации задержек и принципы построения эволюционных алгоритмов; разработать графический интерфейс с вводом данных вручную, из файла и случайной генерацией; реализовать эволюционный алгоритм самостоятельно, с настраиваемыми параметрами; реализовать пошаговую визуализацию процесса поиска решения и график изменения функции качества по поколениям; протестировать программу.

1 ПОСТАНОВКА ЗАДАЧИ

Практика выполнялась в составе учебной бригады с распределением ролей между участниками (разработка алгоритмического ядра, разработка графического интерфейса, тестирование и оформление отчётной документации). Ниже описаны предметная область, в рамках которой выполнялась практика, и конкретные задачи, поставленные перед бригадой.

1.1 Предметная область

Задачи составления расписаний занимают важное место в исследовании операций и прикладной информатике. На практике они возникают везде, где нужно распределить во времени набор работ с известной длительностью и сроками завершения. Такие задачи нужны, чтобы планировать производство, распределять нагрузку на серверы, организовывать доставку в логистике и управлять проектами с жесткими дедлайнами.

Одной из постановок в этой области является задача минимизации задержек: требуется найти такой порядок выполнения работ, при котором суммарное (или максимальное) превышение установленных дедлайнов будет минимальным. В общем случае данная задача относится к классу NP-трудных, из-за чего точные методы решения (полный перебор, динамическое программирование) становятся неприменимыми при большом числе работ. Поэтому на практике для её решения широко применяются приближённые эвристические и методы, среди которых заметное место занимают генетические алгоритмы — методы оптимизации, имитирующие процессы естественного отбора: формирование популяции решений, их отбор, скрещивание и мутацию с постепенным улучшением качества решений от поколения к поколению.

Помимо алгоритма оптимизации, важной практической задачей является разработка удобного программного инструмента с графическим интерфейсом, позволяющего задавать исходные данные, настраивать параметры алгоритма и, что особенно важно в учебных и демонстрационных целях, визуально наблюдать процесс поиска решения. Подобные инструменты

полезны как для изучения самих алгоритмов оптимизации (демонстрация работы эволюционных методов в учебном процессе), так и для практического подбора параметров алгоритма под конкретную задачу.

1.2 Задачи практики

В рамках практики были поставлены следующие задачи:

1. Изучить постановку задачи минимизации суммарной задержки и подходы к её решению;
2. Изучить принципы работы генетических алгоритмов (кодирование решений, селекция, скрещивание, мутация, функция приспособленности);
3. Спроектировать структуру данных для задач, параметров алгоритма и состояний популяции по поколениям;
4. Реализовать генетический алгоритм самостоятельно, с настраиваемыми пользователем параметрами;
5. Разработать графический интерфейс с вводом данных вручную, из файла и случайной генерацией;
6. Реализовать пошаговую визуализацию процесса поиска решения с возможностью просмотра разных решений в популяции, пропуска шагов и отката назад;
7. Реализовать график изменения качества решения по поколениям;
8. Протестировать алгоритм и программу на разных наборах данных;

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Изучение постановки задачи минимизации суммарной задержки

Была изучена математическая постановка задачи однопроцессорного планирования работ с целью минимизации суммарной задержки. Рассмотрена формальная модель: набор из N работ, каждая из которых характеризуется временем выполнения t_i и дедлайном d_i . Работы выполняются последовательно, без прерываний, на одном исполнителе. Введено понятие задержки отдельной работы как $T_i = \max(0, C_i - d_i)$, где C_i — момент завершения работы при заданном порядке выполнения, а также целевая функция — суммарная задержка $\sum_i T_i$, подлежащая минимизации.

Изучены существующие подходы к решению подобных задач: точные методы (полный перебор, динамическое программирование), применимые лишь при небольшой размерности задачи, и приближённые методы. Установлено, что задача минимизации суммарной задержки в общем случае является NP-трудной, в отличие от задачи минимизации максимальной задержки, для которой существует точный полиномиальный алгоритм (правило EDD). Это обусловило выбор приближённого эволюционного метода в качестве основного инструмента решения.

2.2. Изучение принципов работы генетических алгоритмов

Были изучены общие принципы построения генетических алгоритмов как метода приближённой оптимизации, основанного на имитации процессов естественного отбора. Рассмотрены следующие основные понятия и компоненты:

1. Кодирование решений — представление кандидатного решения (особи) в виде хромосомы; для задачи планирования работ рассмотрено представление решения в виде перестановки индексов работ;
2. Функция приспособленности (fitness) — способ оценки качества решения, в данном случае — значение суммарной задержки, соответствующей порядку работ, заданному особью;

3. Селекция — способы отбора родительских особей для формирования следующего поколения, в частности изучена турнирная селекция;
4. Скрещивание (crossover) — операторы формирования потомков на основе родительских особей; для решений, представленных перестановками, изучен оператор Order Crossover (OX), сохраняющий корректность перестановки у потомка;
5. Мутация — оператор случайного изменения особи для поддержания разнообразия популяции и предотвращения преждевременной сходимости; рассмотрен оператор перестановки двух случайных элементов (swap mutation).

Также изучены вспомогательные механизмы, применяемые в генетических алгоритмах: элитизм (сохранение лучших особей без изменений при переходе к новому поколению) и ранняя остановка алгоритма при отсутствии улучшения решения в течение заданного числа поколений (stagnation limit).

2.3. Спроектировать структуру данных для задач, параметров алгоритма и состояний популяции по поколениям

Для реализации генетического алгоритма была спроектирована отдельная структура данных, обеспечивающая хранение входных параметров, промежуточных состояний и итоговых результатов работы алгоритма. Проектирование выполнялось с учетом последующей интеграции с графическим интерфейсом, в котором требуется не только получение окончательного решения, но и пошаговая визуализация процесса эволюции популяции.

Для представления одной задачи была введена структура Task, содержащая идентификатор задачи, длительность выполнения и срок завершения. Такая модель позволяет хранить минимально необходимый набор данных для построения расписания и вычисления суммарной задержки.

Для хранения параметров генетического алгоритма была разработана структура GAConfig. В нее были включены размер популяции, число

поколений, вероятность мутации, вероятность скрещивания, число элитных особей, размер турнира для селекции, а также параметры выбора методов селекции, скрещивания и мутации. Дополнительно были предусмотрены поля для задания начального зерна генератора случайных чисел, ограничения по стагнации и режима сохранения истории. Такое решение позволило отделить конфигурацию алгоритма от его логики и обеспечить удобное изменение параметров без переработки основного кода.

Для представления одной особи популяции была использована структура `Individual`, содержащая хромосому в виде перестановки индексов задач и кэшированное значение приспособленности. В качестве хромосомы была выбрана именно перестановка, так как задача сводится к поиску наилучшего порядка выполнения работ на одном процессоре.

Для хранения информации о конкретном решении в форме, удобной для визуализации, была введена структура `SolutionSnapshot`. В ней сохраняются порядок задач, времена завершения, значения задержек по каждой задаче и суммарная задержка. Наличие такой структуры позволяет избежать повторных вычислений в интерфейсной части программы.

Для представления состояния алгоритма на каждом поколении была разработана структура `GenerationState`. В ней сохраняются номер поколения, вся текущая популяция, лучшее решение в поколении, глобально лучшее решение на текущий момент, худшее значение функции, среднее значение по популяции, массив значений приспособленности и история изменения лучшего результата. Такая организация данных обеспечивает возможность пошагового отображения процесса работы алгоритма, построения графиков и возврата к любому предыдущему поколению.

В результате была получена согласованная система структур данных, охватывающая все основные сущности задачи: входные данные, параметры алгоритма, отдельные решения и историю эволюции популяции. Спроектированная модель стала основой для последующей реализации генетического алгоритма и его интеграции с графическим интерфейсом.

2.4. Реализовать генетический алгоритм самостоятельно, с настраиваемыми пользователем параметрами

Для решения задачи минимизации суммарной задержки был реализован собственный генетический алгоритм без использования готовых специализированных библиотек. Реализация была выполнена в виде отдельного программного модуля, содержащего всю основную логику формирования популяции, оценки решений, отбора, скрещивания, мутации и перехода между поколениями.

В качестве представления решения была выбрана хромосома в виде перестановки задач. Такой подход соответствует постановке задачи однопроцессорного расписания, в которой требуется определить наилучший порядок выполнения работ. Для каждой особи вычислялись времена завершения задач, индивидуальные значения задержек и суммарная задержка, используемая в качестве целевой функции. Чем меньше данное значение, тем более качественным считается решение.

Работа алгоритма начиналась с генерации начальной популяции, состоящей из случайных перестановок всех задач. После этого для каждой особи рассчитывалось значение приспособленности, и формировалось начальное состояние популяции. Далее на каждом поколении выполнялся переход к новой популяции. Сначала без изменений переносилась элитная часть лучших решений, что позволяло сохранять уже найденные удачные варианты. Остальная часть популяции формировалась за счет селекции родительских особей, применения оператора скрещивания и последующей мутации потомков.

В качестве базового метода отбора была реализована турнирная селекция. Ее использование позволило обеспечить достаточно простой и эффективный механизм выбора более качественных особей для размножения. Для скрещивания перестановок был реализован оператор Order Crossover, корректно сохраняющий структуру перестановки без повторений и потерь элементов. В качестве стандартной мутации была выбрана swap mutation, при

которой две случайные задачи меняются местами. Дополнительно были реализованы расширения в виде PMX-скрещивания и inversion mutation, что позволило сделать алгоритм более гибким и пригодным для дальнейших экспериментов с параметрами.

Все основные характеристики алгоритма были вынесены в настраиваемую конфигурацию. Пользователь может задавать размер популяции, число поколений, вероятность мутации, вероятность скрещивания, число элитных особей, размер турнира, используемые методы скрещивания и мутации, начальное значение генератора случайных чисел и ограничение на число поколений без улучшения результата. Благодаря этому реализация не является жестко фиксированной и может использоваться для исследования влияния параметров на качество получаемых решений.

Дополнительно в алгоритм был включен механизм ранней остановки. Если в течение заданного числа поколений не происходило улучшения лучшего найденного результата, выполнение завершалось досрочно. Это позволило сократить лишние вычисления в случаях, когда популяция переставала демонстрировать прогресс.

В результате был реализован самостоятельный генетический алгоритм, полностью соответствующий поставленной задаче и допускающий настройку ключевых параметров пользователем. Созданная реализация обеспечила как получение итогового решения, так и основу для последующего анализа поведения алгоритма в процессе его работы.

2.5. Разработка графического интерфейса с вводом данных вручную, из файла и случайной генерацией

На основе PyQt, библиотеки для работы с фреймворком Qt, был разработан графический интерфейс приложения, реализующий все три предусмотренных способа ввода исходных данных. Основная логика формирования интерфейса реализована в классе MainWindow, где интерфейс разделяется на 3 вкладки с помощью QTabWidget. На вкладке “Источник данных” размещён блок выбора источника с тремя вариантами: ввод вручную,

загрузка из файла и случайная генерация. При выборе варианта отображается соответствующая панель, переключение между которыми осуществляется с использованием виджета QStackedWidget: для ручного ввода - подсказка по редактированию таблицы задач, для загрузки из файла - кнопка выбора файла и поле пути, для случайной генерации - поля параметров генерации.

Под блоком выбора источника размещена таблица задач, формируемая на основе виджета QTableWidget, с колонками “№”, “Время выполнения” и “Дедлайн”, доступная для ручного редактирования значений. Начальное заполнение таблицы выполняется методом `_setup_empty_task_rows()`, а управление её содержимым предусмотрено кнопками «Добавить задачу», «Удалить выбранную» и «Очистить таблицу».

На текущем этапе разработки интерфейс представляет собой нефункциональный макет: элементы ввода и переключения источников данных отображаются и реагируют на действия пользователя визуально, однако обработчики пользовательских действий, подключаемые методом `_connect_signals()`, используют временный метод `poor()`, поэтому кнопки не выполняют обработку данных, а введённые значения не сохраняются и не передаются. Такое решение принято намеренно, чтобы отработать структуру интерфейса до его непосредственной интеграции с алгоритмом.

2.6. Реализация пошаговой визуализации процесса поиска решения

Для отображения хода работы генетического алгоритма используется вкладка «Запуск», содержащая необходимые элементы визуализации. В верхней части вкладки размещены список “История поколений”, позволяющий выбирать и просматривать отдельные поколения (реализован с помощью QListWidget), а также блок “Текущее расписание” с областью диаграммы Ганта и кнопками управления масштабом отображения (“–”, “По ширине”, “+”), для диаграммы Ганта была реализована структура ``GantChartView``.

В нижней части вкладки размещён блок “Информация”, реализованный на основе QGroupBox с полями “Текущее поколение”, “Лучшее значение

Fitness”, “Среднее Fitness”, “Худшее Fitness”, “Суммарная задержка” и “Лучшее расписание”, предназначенными для отображения текущего состояния поиска решения.

Управление процессом визуализации реализовано через набор кнопок: “Старт”, “Пауза”, “Следующий шаг”, “Предыдущий шаг”, “До конца” и “Сброс”. Такой набор элементов управления обеспечивает как пошаговый просмотр эволюции популяции, так и переход к финальному решению без пошагового отображения, а также возврат на предыдущие шаги истории поколений.

Диаграмма Ганта, отображающая расписание задач текущего выбранного решения, вынесена в отдельный виджет (`gantt_chart_view.py`), подготовленный для последующей отрисовки реальных данных. На текущем этапе виджет отображает пустое, поскольку подключение к алгоритму ещё не выполнено.

2.7. Реализация графика изменения качества решения по поколениям

Для отображения динамики изменения функции качества решения в процессе работы алгоритма разработан отдельный виджет графика на основе библиотеки Matplotlib, размещённый в блоке “График функции качества” на вкладке “Запуск”, для данного блока была разработана структура `FitnessPlot`. Виджет подготовлен для отображения зависимости значения функции качества (суммарной задержки) от номера поколения с возможностью пополнения графика новыми точками по мере поступления данных от алгоритма.

На текущем этапе разработки график отображается в виде пустой области с подписанными осями и сеткой, что соответствует состоянию макета до подключения алгоритмической части. Структура виджета предусматривает дальнейшую интеграцию с данными, формируемыми генетическим алгоритмом без необходимости изменения интерфейсной части приложения.

2.8. Протестировать алгоритм и программу на разных наборах данных

После реализации генетического алгоритма было выполнено тестирование программы на различных наборах входных данных. Целью данного этапа являлась проверка корректности вычислений, устойчивости работы алгоритма на типовых и граничных случаях, а также подтверждение готовности программного модуля к дальнейшей интеграции с графическим интерфейсом.

Тестирование проводилось с использованием автоматических модульных тестов, охватывающих как отдельные функции, так и полный цикл работы алгоритма. В ходе проверки контролировалась корректность вычисления целевой функции, правильность формирования поколений, сохранение допустимой структуры решений после применения генетических операторов, воспроизводимость результатов при фиксированном начальном значении генератора случайных чисел, а также корректность сохранения истории поколений.

Для проверки функции оценки использовались небольшие наборы задач, для которых суммарная задержка могла быть рассчитана вручную. Это позволило убедиться в правильности определения моментов завершения задач, индивидуальных задержек и итогового значения целевой функции.

Отдельно проверялась работа алгоритма на стандартных наборах задач из нескольких элементов. В этих случаях анализировалась корректность формирования начальной популяции, перехода между поколениями и накопления истории лучших значений функции приспособленности. Дополнительно контролировалось, что каждая особь после операций скрещивания и мутации по-прежнему представляет собой корректную перестановку задач без повторений и пропусков.

Также были протестированы различные комбинации генетических операторов. Проверка выполнялась как для варианта с Order Crossover и swap mutation, так и для варианта с PMX-скрещиванием и inversion mutation. Это позволило подтвердить корректность работы альтернативных операторов и их совместимость с выбранным представлением решения.

Особое внимание было уделено граничным случаям. Был протестирован сценарий с одной задачей, при котором алгоритм должен корректно работать без ошибок, связанных с невозможностью выполнения стандартных операций скрещивания и мутации. Дополнительно рассматривался случай, когда все задачи могут быть выполнены без нарушения сроков, вследствие чего оптимальное значение целевой функции равно нулю. Проверялся и набор данных с одинаковыми сроками выполнения задач, позволяющий оценить устойчивость алгоритма в условиях повышенной конкуренции между заданиями.

Кроме того, был проверен механизм досрочной остановки. Для этого использовались наборы данных, на которых улучшение решения быстро прекращалось. В результате было подтверждено, что алгоритм корректно завершает работу при достижении заданного предела стагнации и фиксирует причину остановки.

Проверка проводилась и с точки зрения удобства последующей визуализации. Для этого тестировалось сохранение истории состояний по поколениям, включая возможность обращения к любому предыдущему шагу. Такая проверка была необходима для обеспечения поддержки функций пошагового просмотра и возврата к более ранним поколениям в графическом интерфейсе.

По результатам тестирования было установлено, что разработанный алгоритм корректно работает на различных наборах данных, включая стандартные и граничные случаи, сохраняет допустимость решений после генетических преобразований и обеспечивает стабильное формирование истории поколений. Проведенная проверка подтвердила работоспособность программного модуля и его готовность к интеграции с пользовательским интерфейсом.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Для реализации приложения использовался язык программирования Python 3, выбранный в качестве основного инструмента разработки в связи с его широким применением для создания прикладных программ, наличием удобных средств для работы со структурами данных и возможностью быстрой реализации алгоритмов оптимизации.

Основная логика генетического алгоритма была реализована средствами стандартной библиотеки Python. Для описания структур данных использовался модуль `dataclasses`, позволивший компактно и наглядно представить сущности задачи, параметры генетического алгоритма, состояния поколений и итоговые результаты работы программы. Для аннотирования типов применялись средства модуля `typing`, что сделало код более понятным и упростило его сопровождение. Для генерации случайных значений, необходимых при формировании начальной популяции, селекции, скрещивании и мутации, использовался модуль `random`.

Для построения графического пользовательского интерфейса изучена и использована библиотека `PyQt6`, средствами которой реализованы главное окно приложения, система вкладок, формы ввода, таблицы, кнопки, списки, а также панели и группы элементов управления. Для отображения расписания выполнения задач на основе возможностей `PyQt6` разработан отдельный виджет диаграммы Ганта. Для построения графика функции качества решения изучена и использована библиотека `Matplotlib`, интегрированная в интерфейс приложения как отдельный виджет, поддерживающий обновление данных в реальном времени.

Для тестирования разработанного решения использовался фреймворк `pytest`. С его помощью организована автоматическая проверка корректности вычисления целевой функции, формирования популяции, работы генетических операторов, обработки граничных случаев и механизма ранней остановки. Использование данного инструмента позволило повысить

надёжность реализации и упростить повторную проверку программы после внесения изменений.

В качестве среды запуска и отладки применялся интерпретатор Python 3, а запуск программы и тестов выполнялся из консольной среды `cmd`, которая использовалась для выполнения команд запуска модулей, проверки результатов тестирования и общего контроля хода разработки.

Для хранения и редактирования исходного кода использовались текстовые файлы формата `.py`, содержащие модуль генетического алгоритма и модуль автоматических тестов. Реализация была разделена на логические части, что упростило чтение кода, тестирование и последующую интеграцию с графическим интерфейсом.

Таким образом, в рамках практики были использованы Python 3, стандартная библиотека языка (`dataclasses`, `typing`, `random`), библиотеки `PyQt6` и `Matplotlib`, фреймворк `pytest` и консольная среда `cmd`. Указанные технологии обеспечили реализацию генетического алгоритма и графического интерфейса, проверку корректности алгоритма и подготовку программного модуля к дальнейшему использованию.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв

о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(аяся) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

Заключение содержит краткое описание результатов по каждой задаче, обозначенной в разделе «Постановка задачи», результаты работы в целом, согласно обозначенной цели и пр.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.

ПРИЛОЖЕНИЕ А